



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv

CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Sesión 01 — Producto Mínimo Viable (MVP)

2º CFGS DAW/DAM · Proyecto Intermodular

Autores: Sergio García, Guillermo Garrido y Alfredo Oltra



La sesión en un mapa

Recorreremos el camino completo de una idea: **qué es un proyecto** y por qué se gestiona, **qué es un MVP**, cómo **iniciar** la idea (Lean Canvas y Visión), cómo **planificar** su construcción (Backlog, MoSCoW, Sprints) y cómo **construir, medir y lanzar** con el equipo adecuado.



Fundamentos

Proyecto y metodología



El MVP

Qué es y qué no es



Iniciación

Lean Canvas y Visión



Planificación

Backlog, MoSCoW, Sprints



Construir y medir

Ciclo, pivote y cierre



Hilo conductor: el caso práctico **TaskFlow** — una app web (DAW) y su app móvil (DAM) que comparten la misma API.

¿Qué es un proyecto?

Definición: conjunto de actividades **temporales** diseñadas para crear un producto, servicio o resultado **único**.



Temporalidad

Inicio y fin
definidos



Único

No es una tarea
rutinaria



Incertidumbre

Implica un cierto
riesgo



Recursos limitados

Dinero, tiempo,
personas,
tecnología



Ejemplos: diseñar una casa, crear un medicamento, digitalizar el kiosco de tu tío. En informática: desarrollo de software, integración de sistemas, implantación e infraestructura.



El ciclo de vida de un proyecto

Un proyecto no se hace de golpe: se desarrolla **por fases**.

- 1 **Inicio** — se define la idea, los objetivos y la viabilidad.
- 2 **Planificación** — se desglosan tareas, recursos, calendario y presupuesto.
- 3 **Ejecución** — el equipo desarrolla el producto o servicio.
- 4 **Seguimiento y control** — se monitorizan avances, riesgos y desviaciones.
- 5 **Cierre** — entrega final, validación con el cliente y documentación.

i Ejemplo rápido — el kiosco de tu tío: **Inicio**: decide digitalizarse. **Planificación**: qué necesita (TPV, control de revistas, proveedores). **Ejecución**: se instala la app. **Seguimiento**: ajustes tras las primeras semanas. **Cierre**: entrega y formación.



¿Por qué gestionar un proyecto?

Las ganas de lanzar la idea no sustituyen a la gestión del día a día: un proyecto **no gestionado tiende al caos**.



Coordinar recursos

Equipos, tiempo y dinero bajo control



Prever riesgos

Retrasos, cambios y fallos técnicos



Evitar sobrecostes

Proyectos que gastan más de lo previsto



Comunicar y asegurar calidad

Cliente, analistas y devs alineados



No es burocracia: es la diferencia entre entregar o no entregar.



¿Qué es una metodología?

Un conjunto organizado de **principios, procesos, técnicas y herramientas** que guía el proyecto de principio a fin: el **camino estructurado** que convierte una idea en un resultado sin improvisar.

Aporta	Detalle
Estandarización	Todo el equipo sabe qué pasos seguir
Fases	Inicio, planificación, ejecución, seguimiento, cierre
Roles	Especifica quién hace qué
Técnicas y herramientas	Gantt, Kanban, reuniones diarias, entregables
Gestión de riesgos	Anticipa problemas antes de que ocurran

Tipos de metodologías



Tradicionales (cascada)

Lineales y secuenciales: primero se planifica todo, luego se ejecuta.



Híbridas

Planificación y control de riesgos (tradicionales) + iteración y adaptación (ágiles).



Ágiles

Iteraciones cortas, entregas frecuentes y adaptación al cambio. Las más utilizadas hoy.



Concepto común a todas: el MVP

Sea cual sea la metodología, el MVP valida la idea lo antes posible.



El MVP (Minimum Viable Product)

El **producto mínimo viable** es la versión más simple de un producto que permite:

- 📦 **Entregar valor real** a los usuarios (aunque sea limitado).
- 🌍 **Ser usado en condiciones reales.**
- 💬 **Obtener feedback rápido** del mercado.
- 🪶 Con el **mínimo esfuerzo y coste** posible.



No es un prototipo en papel ni una idea teórica: es algo que el cliente puede usar de verdad, aunque tenga pocas funciones.



¿Qué es y qué no es un MVP?



SÍ es un MVP

Producto funcional mínimo que resuelve un problema real y se puede usar hoy mismo.



NO es un MVP

Un prototipo en papel, una idea teórica o el 15% del producto a medias.

*El MVP no es construir el 15% del producto. Es construir el **100% del 15% del producto.***



Telegram: el MVP que retó a WhatsApp



Telegram (2013)

- **Problema:** mensajería rápida, gratuita y multi-dispositivo, sin SMS
- **MVP:** registro con teléfono + chat de texto en la nube; sin stickers, bots ni llamadas
- **Métrica:** ¿suficiente valor para competir con WhatsApp?

Qué validó y qué no tocó

-  Validado: la cuenta funciona a la vez en móvil y escritorio
-  Dejado para después: stickers, bots, llamadas, canales
-  Cada función nueva llegó cuando la base ya usaba lo anterior



El MVP de Telegram ya resolvía lo que WhatsApp no permitía: la misma cuenta en varios dispositivos. Todo lo demás esperó su turno.



Gris: un videojuego español multiplataforma

Gris (2018) lo desarrolló el estudio español **Nomada Studio** (Barcelona), lo publicó **Devolver Digital** y salió de forma **multiplataforma**: Nintendo Switch, PC y Mac, PlayStation, Xbox e iOS y Android.

El producto y su MVP

- **Problema:** una experiencia artística y emocional jugable
- **MVP:** demo corta — un escenario, saltar y la atmósfera acuarela
- **Métrica:** ¿conmueve lo suficiente para comprarlo?

De la validación al lanzamiento

-  La emoción funcionó: premios y comunidad
-  Después: Switch y PC, luego PS, Xbox y móvil
-  El arte no era «extra»: era el producto



El MVP de Gris no demuestra el juego entero: demuestra que la emoción funciona. Validado eso, la misma base se llevó a todas las plataformas.



El Lean Canvas

Un **plan de negocio de una sola página**: rápido, conciso y centrado en los supuestos de mayor riesgo. Olvida los planes de 50 páginas.



Problema



Solución



**Propuesta de
valor**



Ventaja especial



Clientes



Métricas clave



Canales



**Estructura de
costes**



Flujo de ingresos



Se rellena empezando por el centro (Propuesta de Valor → Clientes → Problema → Solución). Es un documento vivo: se actualiza conforme se aprende del mercado.



El Documento de Visión del MVP

Resumen ejecutivo que **complementa al Lean Canvas**: del borrador de hipótesis al documento limpio con el que todo el equipo rema en la misma dirección.

Parte esencial	Qué define
El Problema	Descripción clara del dolor que se alivia
La Solución Mínima	Lo absolutamente esencial de la primera versión
Objetivos de Aprendizaje	Hipótesis clave a validar con el MVP
Autoridad	Quién tiene la última palabra (Product Owner)

i El objetivo no es "construir un login", sino "aprender si los usuarios prefieren registrarse con Google en lugar de con email". El foco pasa de construir a aprender.



El caso práctico: ecosistema TaskFlow

Un mismo gran producto, dos equipos por especialidad:



DAW

MVP web de TaskFlow: la app y su API
REST



DAM

App móvil que consume la misma API



Los equipos no trabajan aislados: DAM depende de la API de DAW. Es la realidad del desarrollo moderno: equipos interdependientes construyendo un ecosistema.



Caso DAW: la Visión de «TaskFlow Web»

✓ Lo que SÍ construiremos (el MVP):

- Registro y login con email y contraseña
- Crear proyectos con nombre y descripción
- Tablero Kanban de 3 columnas fijas
- Tareas con título + arrastrar y soltar
- Invitación por enlace al proyecto
- API REST básica para el equipo DAM

✗ Lo que NO construiremos:

- Registro con Google/Facebook
- Roles de usuario
- Notificaciones
- Personalización de columnas
- Asignación de tareas



Objetivo de aprendizaje: validar si los equipos prefieren esta herramienta frente a Trello o WhatsApp, y si la API está bien diseñada para soportar más clientes.



El Product Backlog

Validada la idea, toca trazar un **mapa flexible hasta el MVP** (la planificación excesiva es el enemigo de la agilidad). Su herramienta central:

*Lista **única, ordenada y viva** de todo lo que se podría desear en el producto. Es el cerebro del proyecto.*

- Cada elemento es una **User Story** que aporta valor al usuario final.
- Al principio contiene **todas** las ideas, funcionalidades y mejoras.

Es como la lista de deseos para los Reyes Magos: infinita, de todo y sin orden. La priorización es el trabajo de los padres: decidir qué es esencial este año.



Priorización MoSCoW: el arte de decir «No»

Nivel	Qué significa	Detalle
M · Must Have	Debe tener	No negociable: es el corazón del MVP
S · Should Have	Debería tener	Importante, no vital: justo después del MVP
C · Could Have	Podría tener	Deseable: «guindas del pastel» si sobra tiempo
W · Won't Have	No tendrá	Acuerdo explícito de no hacer: libera de falsas expectativas



Regla simple: **el MVP está compuesto exclusivamente por los «Must Have».**



Scope Creep: añadir funciones sin control hasta que el proyecto nunca se entrega. MoSCoW es la vacuna — decidir la «W» es tan importante como la «M».



Planificación de Sprints

Un **Sprint** es un período corto y fijo (1–4 semanas) en el que el equipo construye un **incremento de producto funcional**. Sustituye al Gantt a largo plazo.

- 1 **Planificar** — se eligen ítems del Backlog priorizado.
- 2 **Construir** — se trabaja el incremento funcional.
- 3 **Mostrar** — demo del incremento y feedback.
- 4 **Adaptar** — el plan se ajusta para el siguiente Sprint.



Herramientas: Trello para empezar (simple, perfecto para MVPs); Jira, Notion o Azure DevOps en equipos.



Caso DAW: TaskFlow Web en MoSCoW

Must Have (el MVP):

- Registro y login con API REST
- Crear proyectos (/projects) y tareas (/tasks)
- Mover tareas entre las 3 columnas
- Invitación por enlace
-  Documentación de la API

Should Have: asignar tareas · fechas · comentarios

Could Have (futuro):

- Integración con Google Calendar
- Reportes de productividad

Won't Have (no ahora):

- Diagramas de Gantt
- App móvil nativa (eso lo hace DAM)
- Sistema de facturación



El Must Have de DAW incluye la documentación de la API: no construye solo para usuarios web, sino para el equipo DAM que la consumirá.

El bucle Construir → Medir → Aprender

El motor del proyecto: un ciclo rápido que **maximiza el aprendizaje y minimiza el desperdicio**.

- 1 **Construir** — transformar la planificación en producto tangible, por incrementos.
- 2 **Medir** — KPIs de comportamiento real: ¿se registran? ¿usan la funcionalidad? ¿cómo interactúan?
- 3 **Aprender** — decidir con datos (cuantitativos y cualitativos): pivotar o perseverar.



Ejecución: la fábrica de software (CI/CD)

La **Integración y Despliegue Continuo** automatiza integrar, probar y desplegar cambios **en minutos y de forma segura**.

- Reduce el **coste de la experimentación**: probar ideas en producción en horas, no semanas.
- Es el motor que acelera el ciclo de aprendizaje y la validación de hipótesis.



CI/CD no es solo automatizar tareas: acelera dramáticamente el aprender de los usuarios reales.



En TaskFlow: GitHub Actions ejecuta tests en cada commit; el 85% de las peticiones a la API vienen de la app de DAM (valida la multiplataforma) y «mover tareas» iba lento → se optimizó la API.



Caja de herramientas del ciclo

Herramienta	Para qué
 GitHub Actions	CI/CD integrado con GitHub; gratis en proyectos públicos
 GitLab CI	Pipeline visual; Docker integrado
 Jenkins	El más configurable; self-hosted
 Google Analytics	Comportamiento del usuario en web
 Prometheus + Grafana	Métricas de sistema, dashboards y alertas
 Firebase Analytics	Apps móviles y eventos personalizados



DAW: GitHub Actions + Google Analytics + Grafana · DAM: GitHub Actions + Firebase + Crashlytics.



El arte del pivotaje estratégico

*Pivotar no es un ajuste táctico: es un **cambio de rumbo estratégico fundamentado en datos concretos.***



Pivotar

Cambiar de dirección cuando los datos demuestran que la hipótesis no funciona.



Perseverar

Seguir el rumbo cuando los KPIs validan la hipótesis inicial.



Caso emblemático: ****Slack**** nació de un videojuego que nadie usaba; su herramienta de chat interna era lo valioso. Pivotaron y hoy es la referencia en comunicación de equipos.

El lanzamiento: cerrar el proyecto, abrir el producto

El cierre del proyecto MVP no es un final: es el comienzo de la vida del producto.

La actividad clave es entregar el MVP a los **primeros usuarios**. Internamente, el paso más crucial es la **retrospectiva de lecciones aprendidas**.

 Todo el aprendizaje se recoge en un **Informe de Lanzamiento**: define los siguientes pasos del producto basándose en lo aprendido.



La retrospectiva: «autopsia sin culpas»

*El objetivo no es buscar culpables, sino analizar **qué falló en el proceso** para mejorarlo en el siguiente Sprint.*

El equipo analiza:

-  Qué funcionó
-  Qué no funcionó
-  Qué se aprendió del producto, de los usuarios y del propio proceso

 Si la retrospectiva se convierte en una caza de culpables, deja de servir: el foco es el proceso, no las personas.

 Caso DAW: 15 equipos la primera semana y 2.000 llamadas diarias a la API (80% desde móvil). La siguiente función («fechas de entrega», un Should Have) irá primero a la API para que DAM la aproveche.



La Triple Restricción

Un malabarista con tres bolas: **Alcance**, **Tiempo** y **Costo**, con la **Calidad** como base que sostiene todo. No puedes hacer las tres bolas grandes a la vez: el MVP decide conscientemente que la del **Alcance** sea muy pequeña.



TIEMPO
FIJO



COSTO
CONTROLADO



ALCANCE
VARIABLE



CALIDAD
NO NEGOCIABLE



Estrategia: **sacrificar el ALCANCE** para proteger TIEMPO, COSTO y CALIDAD. En TaskFlow: 5-6 funciones clave en 4 semanas, con hosting gratuito y la API con tests de integración.

La calidad no es negociable

Qué significa «calidad no negociable»:

- Código **limpio y mantenible**
- Excelente experiencia en las pocas funciones que hay
- No perfecto, pero **tampoco una chapuza** que haya que tirar en la próxima versión

La brújula ante la presión: ⬇ «Falta tiempo» · ⬇ «Se agota el presupuesto» · ⬇ «Queremos más funciones» → en los tres casos, recortar alcance · 💎 «Esto no funciona bien» → **la calidad NUNCA se negocia**

⚠ Un MVP es un producto **enfocado**, no un producto con errores. Lo poco que se construye debe ser robusto y profesional.



Los roles de un equipo técnico moderno

*Un equipo técnico **no es solo un grupo de programadores**: es un ecosistema de roles especializados que colaboran para construir, lanzar y mantener el producto.*



**Liderazgo y
estrategia**



**Desarrollo de
software**



**Infraestructura y
DevOps**



**Calidad y
producto**



Liderazgo y desarrollo

Liderazgo y estrategia

- **CTO** — estrategia tecnológica a largo plazo
- **Product Manager** — el **qué** y el **porqué**; prioriza el Backlog (MoSCoW)
- **Project Manager** — el **cómo** y el **cuándo**; guardián de la Triple Restricción
- **Engineering Manager** — personas, cultura y crecimiento del equipo

Desarrollo de software

- **Tech Lead** — arquitectura y calidad técnica
- **Senior** — diseña sistemas complejos y guía
- **Mid-Level** — autónomo; la columna vertebral
- **Junior** — aprende con mentoría y gana experiencia



Especializaciones comunes: backend, frontend, móvil y full-stack.

DevOps y calidad

Infraestructura y operaciones

- **DevOps Engineer** — pipelines CI/CD e infraestructura como código
- **SRE** — fiabilidad y uptime; «automatizar para prevenir» (Prometheus/Grafana)
- **Cloud Engineer** — infraestructura en la nube segura y eficiente

Calidad y producto

- **QA Engineer** — diseña y ejecuta pruebas para cazar errores antes que los usuarios
- **UX Designer** — cómo se siente el producto: flujos e investigación
- **UI Designer** — cómo se ve: colores, tipografía, composición

 QA conecta directamente con la Triple Restricción: es el guardián del pilar de la calidad.



Chequeo rápido de la sesión

1. ¿Qué tres rasgos definen a todo proyecto?
2. ¿Puede un prototipo en papel ser un MVP? ¿Por qué?
3. ¿Qué bloque del Lean Canvas se rellena primero y por qué?
4. ¿Qué letra de MoSCoW forma el MVP y qué aporta la «W»?
5. ¿Qué restricción se sacrifica en un MVP y cuál nunca se negocia?
6. ¿Qué diferencia hay entre Product Manager y Project Manager?



Resumen de la sesión

- Un **proyecto** es temporal, único y con recursos limitados; se desarrolla **por fases** y sin gestión tiende al caos.
- Una **metodología** estructura el trabajo; el **MVP** valida la idea: el 100% del 15% del producto.
- La **iniciación** se hace con el **Lean Canvas** y el **Documento de Visión** (foco en aprender, no en construir).
- La **planificación ágil** usa Backlog, **MoSCoW** (el MVP = Must Have) y **Sprints** con incremento funcional.
- El ciclo **Construir** → **Medir** → **Aprender** decide si **pivotar** (Slack) o **perseverar**; el lanzamiento abre la vida del producto con una retrospectiva sin culpas.
- La **Triple Restricción** sacrifica alcance para proteger tiempo, coste y **calidad no negociable**, con roles especializados en liderazgo, desarrollo, DevOps y calidad.

